

Lecture 14: Binary Search in Array

Is lecture mein hum **Binary Search** ke advanced variations cover karenge. Basic Binary Search se aage badhte hue hum **First Occurrence**, **Last Occurrence**, aur **Search Insert Position** jaise problems solve karenge. Ye variations interviews mein **sabse zyada puche jate hain** aur inka template rote learn hona chahiye har developer ko.



Binary Search Template Recap

PATTERN

Binary Search — Divide & Conquer, Sorted Array, Logarithmic Time

Standard Binary Search

Binary Search ka basic template hum Lecture 13 mein already cover kar chuke hain. Lekin advanced variations ke liye template ko thoda modify karna padta hai. Pehle standard template ko recall karte hain:

```
// Standard Binary Search Template
int binarySearch(vector<int>& nums, int target) {
    int low = 0;
    int high = nums.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }

    return -1;
}
```

Key Insight: Advanced Binary Search problems mein **target milne ke baad bhi search karna padta hai**. First Occurrence mein left side search karo, Last Occurrence mein right side. Ye standard template se alag hai jahan target milte hi return kar dete hain.

Problem 1: Find First and Last Position of Element

LeetCode 34: Find First and Last Position of Element in Sorted Array

Difficulty: Medium | Binary Search | First & Last Occurrence

Problem Statement:

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value. If target is not found in the array, return `[-1, -1]`.

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- `nums` is a non-decreasing array
- $-10^9 \leq \text{target} \leq 10^9$

// Example 1

Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 8` Output: `[3, 4]` //

Explanation: 8 appears at index 3 and 4

// Example 2

Input: `nums = [5, 7, 7, 8, 8, 10]`, `target = 6` Output: `[-1, -1]` //

Explanation: 6 is not present in the array

// Example 3 (Empty Array)

Input: `nums = []`, `target = 0` Output: `[-1, -1]`

// Example 4 (Single Element)

Input: `nums = [1]`, `target = 1` Output: `[0, 0]`

Introduction

Is problem mein humein sorted array mein **target ka first occurrence** aur **last occurrence** dono dhoondhna hai. Normal Binary Search target milte hi return kar deta hai, lekin yahan humein **boundary positions** chahiye. Isliye hum **do alag Binary Searches** lagayenge: ek first occurrence ke liye, ek last occurrence ke liye.

The Analogy

Socho aapke paas ek library mein books hain alphabetically sorted. Aapko ek specific book dhoondhni hai aur pata karna hai ki wo **pehli baar kahan rakhi hai** aur **last baar kahan rakhi**

hai. Pehli search mein aap left side check karte ho, last search mein right side. Exactly aise hi do Binary Searches kaam karte hain.

How It Works Internally

1 First Occurrence Search: Jab target mil jaye, us index ko save karo aur **left half** mein search karna continue karo. Kyunki first occurrence left side mein hi hoga.

2 Last Occurrence Search: Jab target mil jaye, us index ko save karo aur **right half** mein search karna continue karo. Kyunki last occurrence right side mein hoga.

3 Return: Dono searches ke results ko combine karke {first, last} return karo.

WORKFLOW

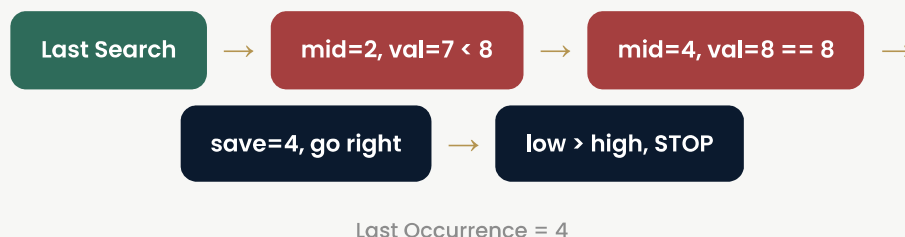
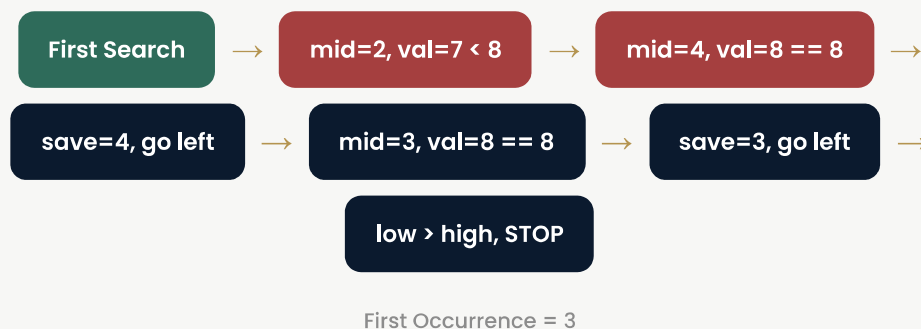
First Search: target == nums[mid] ? save index, go LEFT (high = mid - 1).

Last Search: target == nums[mid] ? save index, go RIGHT (low = mid + 1).

Key Difference: Target milne ke baad bhi search continue karna hai.

Visual Walkthrough

Array: [5, 7, 7, 8, 8, 10], **Target:** 8



Code Implementation

```

// LeetCode 34: Find First and Last Position
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {

        int first = -1;
        int last = -1;

        // =====
        // Find First Occurrence
        // =====
        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (nums[mid] == target) {
                first = mid;
                // Search towards left
                high = mid - 1;
            }
            else if (nums[mid] < target) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        // =====
        // Find Last Occurrence
        // =====
        low = 0;
        high = nums.size() - 1;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (nums[mid] == target) {
                last = mid;
                // Search towards right
                low = mid + 1;
            }
            else if (nums[mid] < target) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }

        return {first, last};
    }
};

```

Complexity Analysis

Case	Time Complexity	Reason
First Search	$O(\log n)$	Binary Search on sorted array.
Last Search	$O(\log n)$	Binary Search on sorted array.
Total	$O(\log n)$	Two independent Binary Searches.
Space	$O(1)$	Sirf first, last, low, high, mid variables.

IMPORTANT

Target milne ke baad bhi search karna hai. First Occurrence mein $high = mid - 1$ karo (left jao), Last Occurrence mein $low = mid + 1$ karo (right jao). Ye standard Binary Search se alag hai jahan return mid karte hain.

EDGE CASE

- **Empty array:** Directly return $[-1, -1]$.
- **Single element:** Agar target match kare toh $[0, 0]$ return hoga.
- **All same elements:** First = 0, Last = $n-1$ return hoga.
- **Target not present:** $[-1, -1]$ return hoga.

INTERVIEW TIP

Ye problem **interview mein bahut common** hai. Hamesha **do alag functions** likhne ka socho: `findFirst()` aur `findLast()`. Isse code modular aur readable hota hai. Interviewer ko batana ki aap **template-based approach** follow karte ho — same Binary Search template ko modify karke different variations solve karte ho.

KEY TAKEAWAYS

- First Occurrence ke liye target milne par **left side** search karo.
- Last Occurrence ke liye target milne par **right side** search karo.
- Total time $O(\log n)$, space $O(1)$.
- Ye template **Lower Bound** aur **Upper Bound** problems mein bhi use hota hai.



Problem 2: Search Insert Position

LeetCode 35: Search Insert Position

Difficulty: Easy | Binary Search | Lower Bound

Problem Statement:

Given a sorted array of distinct integers `nums` and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order. You must write an algorithm with **$O(\log n)$** runtime complexity.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` contains distinct values sorted in ascending order
- $-10^4 \leq \text{target} \leq 10^4$

// Example 1 (Target Found)

Input: `nums = [1, 3, 5, 6]`, `target = 5` Output: `2` // Explanation: 5 is present at index 2

// Example 2 (Target Not Found - Insert in Middle)

Input: `nums = [1, 3, 5, 6]`, `target = 2` Output: `1` // Explanation: 2 should be inserted at index 1 to maintain sorted order

// Example 3 (Target Not Found - Insert at End)

Input: `nums = [1, 3, 5, 6]`, `target = 7` Output: `4` // Explanation: 7 should be inserted at the end (index 4)

// Example 4 (Target Not Found - Insert at Beginning)

Input: `nums = [1, 3, 5, 6]`, `target = 0` Output: `0` // Explanation: 0 should be inserted at the beginning (index 0)

Introduction

Is problem mein humein **Lower Bound** dhoondhna hai. Lower Bound wo **pehla index** hai jahan target insert kiya ja sakta hai taaki array sorted rahe. Agar target already present hai, toh uska index return karo. Nahi toh us position ka index return karo jahan target ko insert karna chahiye.

Key Insight: Search Insert Position actually **Lower Bound** find karna hai. Lower Bound = pehla element jo target se **greater than or equal** hai. Iska matlab agar target present hai toh uska index, nahi toh usse just bade element ka index.

The Analogy

Socho aapko ek class mein baithna hai roll number ke hisaab se. Aapka roll number 25 hai. Class mein 10, 20, 30, 40 roll numbers hain. Aap 20 ke baad aur 30 ke pehle baithoge — matlab **index 2**

pe. Ye exactly Lower Bound ka concept hai.

How It Works Internally

- 1 Initialize:** $low = 0, high = n - 1, index = n$ (default end mein insert karne ka position).
- 2 Binary Search:** mid calculate karo aur compare karo.
- 3 Target Found:** $nums[mid] == target$? Directly return mid.
- 4 Target Greater:** $nums[mid] < target$? Right half mein jao: $low = mid + 1$.
- 5 Target Smaller:** $nums[mid] > target$? Ye possible insertion position hai. $index = mid$ save karo aur left half mein jao: $high = mid - 1$.
- 6 Return:** Loop end hone par index return karo.

WORKFLOW

Condition: $nums[mid] > target$? $index = mid, high = mid - 1$.

Logic: Jab bhi $nums[mid]$ target se bada mile, wo possible insertion position hai.

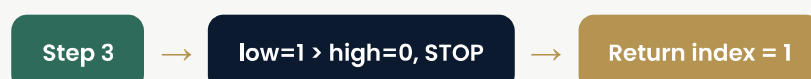
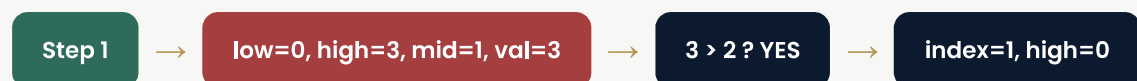
Aur bhi chhota index mil sakta hai, isliye left side search karte raho.

Visual Walkthrough

Array: [1, 3, 5, 6], **Target:** 2



Indices: 0 1 2 3 | Target = 2 should be at index 1



2 should be inserted at index 1

Code Implementation

```
// LeetCode 35: Search Insert Position
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {

        int low = 0;
        int high = nums.size() - 1;

        // Default insertion position is at the end
        int index = nums.size();

        while (low ≤ high) {

            int mid = low + (high - low) / 2;

            // Target found
            if (nums[mid] == target) {
                return mid;
            }

            // Search in right half
            else if (nums[mid] < target) {
                low = mid + 1;
            }

            // Possible insertion position
            else {
                index = mid;
                high = mid - 1;
            }
        }

        return index;
    }
};
```

Complexity Analysis

Case	Time Complexity	Reason
Best	$O(1)$	Target first mid pe hi mil jaye.
Average	$O(\log n)$	Binary Search halves the search space each step.
Worst	$O(\log n)$	Target not present, full Binary Search required.
Space	$O(1)$	Sirf low, high, mid, index variables.

IMPORTANT

Default index = n initialize karna mat bhoolna. Agar target sabse bada hai, toh wo end mein insert hoga. Agar aap index = n nahi initialize karte, toh galat answer aa sakta hai. Ye edge case **interviews mein test hota hai**.

EDGE CASE

- **Target smallest:** index = 0 return hoga.
- **Target largest:** index = n return hoga (default).
- **Target present:** Direct index return hoga.
- **Single element:** Properly handle karta hai.

INTERVIEW TIP

Search Insert Position **Lower Bound** ka classic example hai. Interviewer hamesha $O(\log n)$ solution expect karta hai. Linear Search mat batana. Is problem ka concept **C++ STL lower_bound()** aur **Java Arrays.binarySearch()** mein bhi use hota hai. Ye template **Floor** aur **Ceil** problems mein bhi directly apply hota hai.

KEY TAKEAWAYS

- Search Insert Position = **Lower Bound** find karna.
- Default index = n initialize karo (target sabse bada ho sakta hai).
- $\text{nums}[\text{mid}] > \text{target}$ hone par index = mid save karo aur left jao.
- Time $O(\log n)$, Space $O(1)$.
- Ye template **Floor**, **Ceil**, aur **Upper Bound** mein bhi use hota hai.



Comparative Summary: Binary Search Variations

Problem	Core Concept	When Target Found	Return Value	Time
Standard Binary Search	Find any occurrence	Return immediately	mid index	$O(\log n)$
First Occurrence	Leftmost position	Save index, go LEFT	saved first index	$O(\log n)$
Last Occurrence	Rightmost position	Save index, go RIGHT	saved last index	$O(\log n)$
Search Insert Position	Lower Bound	Return immediately	insertion index	$O(\log n)$

LECTURE TAKEAWAYS

- **Binary Search** ka basic template modify karke multiple variations solve kiye ja sakte hain.
- **First Occurrence** mein target milne ke baad **left side** search karo ($high = mid - 1$).
- **Last Occurrence** mein target milne ke baad **right side** search karo ($low = mid + 1$).
- **Search Insert Position** = **Lower Bound** find karna. Default index = n rakhna mat bhoolna.
- Har variation ka **template rote learn** karo. Interviews mein directly apply hoga.
- Code likhne se pehle **dry run** karo. Paper pe chalao, tabhi edge cases catch honge.